Aula 08 — React: componentes, props e estado

Curso de Formação de Programação Aplicada · Módulo 1 · FPA Labs

> **Duração:** 2h · **Pré-requisito:** Aulas 01–03 (JavaScript moderno). Não depende do back-end.

Objetivos de aprendizagem

Ao final desta aula o aluno deve conseguir:

1. Explicar que problema o React resolve em relação ao JavaScript puro.
2. Criar um projeto React com Vite e entender a estrutura de pastas.
3. Escrever componentes com JSX.
4. Passar dados entre componentes com **props**.
5. Criar estado com `useState` e entender por que a tela se atualiza sozinha.
6. Renderizar listas com `.map()` e usar `key` corretamente.

Plano de aula (2h)

Tempo	Bloco	Conteúdo
0:00–0:10	Aquecimento	Recapitular o back-end pronto · o que vem agora
0:10–0:30	Conceito	O problema do DOM manual · componentes · JSX
0:30–1:05	Código ao vivo	`npm create vite` → componente → props → `useState`
1:05–1:15	Intervalo	
1:15–1:50	Mão na massa	Exercícios 1 a 4
1:50–2:00	Fechamento	Imutabilidade no estado · prévia da aula 09

---

1. O problema que o React resolve

No Módulo 0 vocês fizeram coisas assim:

```
const lista = document.querySelector("#lista");
const contador = document.querySelector("#contador");

function adicionarTarefa(texto) {
  const item = document.createElement("li");
  item.textContent = texto;
  lista.appendChild(item);
  contador.textContent = `${lista.children.length} tarefas`;   // não esqueça de atualizar!
}
```

Funciona. Mas repare no problema: **você precisa lembrar de atualizar cada pedaço da tela na mão.** Com 3 elementos, tudo bem. Com 30, você esquece um — e a tela mostra informação errada. Esse tipo de bug é o mais comum em front-end feito à mão.

A virada de chave do React

Em vez de dizer **como mudar** a tela passo a passo, você descreve **como a tela deve ser** para cada estado dos dados. Quando o dado muda, o React se encarrega de atualizar a tela.

```
JavaScript puro:  dado mudou → EU atualizo cada pedaço da tela
React:            dado mudou → o REACT atualiza a tela por mim
```

> **Analogia:** é a diferença entre dirigir com câmbio manual (você troca cada marcha, na hora certa) e automático (você diz para onde quer ir; o carro cuida das marchas). O automático não é mágica — ele só automatizou uma decisão repetitiva que você fazia errado de vez em quando.

---

2. Criando o projeto

```
npm create vite@latest meu-app -- --template react
cd meu-app
npm install
npm run dev
```

O **Vite** é a ferramenta que cria e serve o projeto durante o desenvolvimento. Ele abre em `http://localhost:5173` e recarrega a tela a cada `Ctrl+S`.

> ⚠️ Repare: **duas portas diferentes**. O back-end continua na `3000`; o front roda na `5173`. São dois programas separados. Na aula 09 eles vão conversar.

A estrutura que importa

```
meu-app/
├── index.html          o único HTML de verdade — quase vazio
├── package.json
└── src/
    ├── main.jsx        o ponto de partida: liga o React ao HTML
    ├── App.jsx         o componente principal
    ├── App.css         estilos
    └── components/     ← pasta que VOCÊ cria, para seus componentes
```

Abra o `index.html`: ele tem só uma `<div id="root">` vazia. Todo o resto da página é criado pelo React em tempo de execução, a partir do `main.jsx`.

Primeira faxina: apague o conteúdo de `App.jsx` e de `App.css` e comece do zero. O template vem cheio de coisa que só atrapalha quem está aprendendo.

---

3. Componentes: as peças de LEGO

Um componente é **uma função que devolve um pedaço de tela**.

```jsx
function Saudacao() {
  return <h1>Olá, turma FPA!</h1>;
}
```

Duas regras obrigatórias:

1. **O nome começa com letra maiúscula.** `Saudacao`, não `saudacao`. É assim que o React distingue seu componente de uma tag HTML comum.
2. **Devolve UM elemento raiz.** Para devolver vários irmãos, embrulhe num `<div>` ou num *fragment* (`<>...</>`).

Usa-se como se fosse uma tag:

```jsx
function App() {
  return (
    <div>
      <Saudacao />
      <Saudacao />   {/* dá para repetir à vontade */}
    </div>
  );
}
```

Por que dividir em componentes? Pelas mesmas razões da aula 06: cada peça tem um nome, uma responsabilidade, e pode ser reaproveitada e testada isoladamente. Um arquivo de 800 linhas de JSX é tão ruim quanto um `server.js` de 800 linhas.

---

4. JSX

JSX é HTML escrito dentro do JavaScript. Não é string, não é template — é sintaxe da linguagem (traduzida pelo Vite antes de chegar ao navegador).

```
const nome = "Maria";
const idade = 28;

function Perfil() {
  return (
    <div className="perfil">
      <h2>{nome}</h2>                    {/* chaves = "aqui entra JavaScript" */}
      <p>Idade: {idade}</p>
      <p>Ano de nascimento: {2026 - idade}</p>
      <p>{idade >= 18 ? "Maior de idade" : "Menor de idade"}</p>
    </div>
  );
}
```

As chaves `{}` são a ponte entre o HTML e o JavaScript. Dentro delas cabe qualquer *expressão* (algo que resulta num valor). Não cabe `if` nem `for` — para isso usamos ternário e `.map()`.

As diferenças em relação ao HTML

HTML	JSX	Por quê
`class="x"`	`className="x"`	`class` é palavra reservada do JavaScript
`for="x"`	`htmlFor="x"`	idem
`onclick="..."`	`onClick={fn}`	camelCase, e recebe uma **função**
` `	` `	toda tag precisa ser fechada
`style="color: red"`	`style={{ color: "red" }}`	é um objeto JavaScript

O erro mais comum: `class` em vez de `className`. O React avisa no console — leia os avisos.

Renderização condicional

```
{carregando && <p>Carregando...</p>}                {/* mostra só se true */}
{erro ? <p className="erro">{erro}</p> : <Lista />} {/* um ou outro */}
```

⚠️ Cuidado com `{lista.length && <Lista />}`: se o tamanho for `0`, o React imprime **"0"** na tela. Use `{lista.length > 0 && <Lista />}`.

---

5. Props: passando dados para baixo

Props são os **parâmetros** de um componente. Quem usa o componente decide os valores.

```
function Card({ titulo, descricao, destaque }) {
  return (
    <div className={destaque ? "card destaque" : "card"}>
      <h3>{titulo}</h3>
      <p>{descricao}</p>
    </div>
  );
}

// usando:
<Card titulo="Node.js" descricao="JavaScript no servidor" destaque={true} />
<Card titulo="React" descricao="Interfaces em componentes" />
```

Detalhes que importam:

- Texto vai entre aspas; **qualquer outra coisa vai entre chaves**: `idade={28}`, `ativo={true}`, `itens={[1, 2]}`, `aoClicar={minhaFuncao}`.
- A desestruturação `({ titulo, descricao })` é a mesma da aula 02.
- **Props são somente leitura.** Um componente nunca altera as props que recebeu. Se precisa mudar algo, ou é estado, ou é trabalho de quem passou a prop.

`children`: o conteúdo entre as tags

```
function Painel({ titulo, children }) {
  return (
    <section className="painel">
      <h2>{titulo}</h2>
      {children}
    </section>
  );
}

<Painel titulo="Minhas tarefas">
  <p>Isto tudo chega como children.</p>
  <button>Novo</button>
</Painel>
```

---

6. `useState`: fazendo a tela reagir

Props vêm de fora e não mudam. Mas e o que muda com o tempo — o texto de um input, um contador, a lista de tarefas? Isso é **estado**.

```
import { useState } from "react";

function Contador() {
  const [contagem, setContagem] = useState(0);
  //     ↑ valor    ↑ função     ↑ valor inicial

  return (
    <div>
      <p>Você clicou {contagem} vezes</p>
      <button onClick={() => setContagem(contagem + 1)}>Clique</button>
    </div>
  );
}
```

O que acontece quando você clica:

```
1. onClick chama setContagem(1)
2. o React guarda o novo valor
3. o React chama a função Contador() DE NOVO
4. agora contagem vale 1, e o JSX sai diferente
5. o React compara com o que estava na tela e atualiza só o que mudou
```

O componente inteiro roda de novo a cada mudança de estado. Estranha no começo, mas é o que torna tudo previsível: a tela é sempre o resultado do estado atual.

As três regras do estado

1. Nunca altere o estado diretamente.

```
contagem = contagem + 1;      // ❌ o React não fica sabendo; a tela não muda
setContagem(contagem + 1);    // ✅
```

2. Para objetos e arrays, crie um valor novo — exatamente a imutabilidade da aula 02:

```
tarefas.push(nova);                 // ❌ mesmo array; o React não vê diferença
setTarefas([...tarefas, nova]);     // ✅ array novo

usuario.nome = "Ana";               // ❌
setUsuario({ ...usuario, nome: "Ana" });  // ✅
```

> Este é **o erro nº 1 de quem começa em React**: mutar o estado e ficar 20 minutos procurando por que "a tela não atualiza". O React compara referências: se o array é o mesmo objeto na memória, ele conclui que nada mudou. Aquela aula sobre spread era sobre isto.

3. Hooks só no topo do componente. Nunca dentro de `if`, `for` ou função aninhada:

```
if (algo) {
  const [x, setX] = useState(0);   // ❌ quebra
}
```

---

7. Listas e a `key`

```
function ListaDeTarefas({ tarefas }) {
  return (
    <ul>
      {tarefas.map((tarefa) => (
        <li key={tarefa.id}>{tarefa.titulo}</li>
      ))}
    </ul>
  );
}
```

O `.map()` da aula 02, agora transformando dados em elementos de tela.

Por que `key`? Quando a lista muda, o React precisa saber qual item é qual, para não redesenhar tudo. A `key` é a identidade de cada item.

- ✅ Use o `id` que vem do banco de dados.
- ❌ **Não use o índice do array.** Se a lista for reordenada ou tiver itens removidos do meio, o React associa o item errado — e você vê bugs bizarros, tipo o texto de um input pulando de linha.

---

8. Eventos

```
<button onClick={aoClicar}>Clique</button>          {/* ✅ passa a função */}
<button onClick={aoClicar()}>Clique</button>        {/* ❌ CHAMA na hora de desenhar */}
<button onClick={() => remover(tarefa.id)}>Apagar</button>  {/* ✅ com argumento */}
```

O segundo caso é um clássico: sem a arrow function, a função executa no momento em que o componente é desenhado, não quando o usuário clica. Se ela mexer no estado, você ganha um loop infinito.

---

9. Ferramentas novas desta aula

`useState(valorInicial)`

O que é: um *hook* — uma função especial do React que só funciona dentro de componentes.

```
const [contagem, setContagem] = useState(0);
//     ↑ o valor  ↑ a função   ↑ o valor inicial
```

Ele devolve um **array de dois itens**, e usamos desestruturação de array (aula 02) para nomeá-los. Os nomes são escolha sua; a convenção é `x` e `setX`.

O que faz	guarda um valor que, ao mudar, redesenha a tela
Quando usar	para tudo que muda com o tempo e aparece na tela
Quando NÃO usar	para valores que dá para **calcular** a partir de outro estado

`.toSorted()` — a versão imutável do `sort`

```
[...lista].sort((a, b) => a - b);    // clássico: copia antes
lista.toSorted((a, b) => a - b);     // moderno: já devolve novo
```

No React, você **nunca** pode usar `.sort()` direto no estado — ele altera o array original, e aí a tela não atualiza. Use uma das duas formas acima.

`.map()` com `key`

```
{tarefas.map((tarefa) => <li key={tarefa.id}>{tarefa.titulo}</li>)}
```

Por que `map` e não `for`? Porque dentro do JSX só cabe **expressão** — algo que resulta num valor. Um `for` é um comando, não uma expressão. O `map` resulta num **array de elementos**, e é exatamente isso que o React sabe desenhar.

Por que `key`? É a identidade de cada item. Quando a lista muda, o React usa a `key` para saber qual item é qual e redesenhar só o necessário. Use o `id`; nunca o índice.

Eventos: `onClick`, `onChange`, `onSubmit`

Em JSX, eventos são **camelCase** e recebem uma **função**, não uma string:

```
<button onClick={aoClicar}>                  {/* ✅ passa a função */}
<button onClick={aoClicar()}>                {/* ❌ CHAMA ao desenhar */}
<button onClick={() => remover(tarefa.id)}>  {/* ✅ com argumento */}
```

O segundo caso é um erro clássico: sem a arrow function, a função executa no momento em que o componente é desenhado. Se ela mexer no estado, você ganha um loop infinito.

`style` como objeto

```
<span style={{ backgroundColor: "red", fontSize: "12px" }}>
```

Duas chaves: as de fora dizem "aqui vem JavaScript", as de dentro são o objeto. E as propriedades viram **camelCase**: `background-color` → `backgroundColor`.

`className` em vez de `class`

`class` é palavra reservada do JavaScript, então o JSX usa `className`. O mesmo vale para `for` em labels, que vira `htmlFor`.

Para classes condicionais, três formas — escolha a que ficar mais legível:

```
<li className={feita ? "item concluida" : "item"}>          // ternário

const classes = ["card"];                                    // array + join
if (destaque) classes.push("destaque");
<div className={classes.join(" ")}>

<div className={`card ${destaque ? "destaque" : ""}`}>       // template literal
```

`&&` e ternário no JSX

```
{carregando && <p>Carregando...</p>}                 // mostra só se true
{erro ? <Erro /> : <Lista />}                         // um ou outro
```

⚠️ **Cuidado com números:** `{lista.length && <Lista />}` imprime **"0"** na tela quando a lista está vazia, porque `0 && algo` resulta em `0` — e o React desenha o zero. Use `{lista.length > 0 && <Lista />}`.

Recapitulação em 3 frases

1. Componente é uma função que devolve tela; props entram de fora e são somente leitura.
2. `useState` cria dados que mudam — e mudar o estado é o que faz o React redesenhar.
3. Nunca mute o estado: crie um valor novo com spread, ou a tela não atualiza.

Tarefa para a próxima aula

Exercícios 1 a 4 de `exercicios/README.md`.

Na próxima aula

Vamos ligar os dois mundos: o React vai buscar os dados na **sua API**, criada nas aulas 04 a 07.
